# Wyszukiwarka katalogowa

## Kontekst

Fikcyjna firma **Orbita Archiwum** korzysta ze skryptu `main.py`, który wczytuje `data/satcat.csv` i pokazuje liczbę oraz podgląd rekordów. Firma zleca rozbudowę skryptu, ponieważ analitycy potrzebują listy obiektów, których nazwy zawierają szukaną frazę.

## Podział pracy

### Student 1 - analiza w pandas

1. **Raport podstawowy.** Uzupełnij `analyze`. Program ma wyszukać fragment OBJECT_NAME bez rozróżniania wielkości liter.
2. **Podsumowanie.** Uzupełnij `summarize`. Policz wyniki według OBJECT_TYPE. Dla każdego typu policz liczbę pasujących wierszy.

W obu funkcjach analizy zastosuj dodatkowe filtry `--type` i `--from-year`, jeśli użytkownik je poda.

### Student 2 - polecenia CLI w argparse

1. **Polecenie `report`.** Dodaj podpolecenie z kryterium z przykładu i opcją `--output`. Wywołaj analizę i zapis CSV. Sprawdź poprawność argumentów.
2. **Polecenie `summary`.** Dodaj podpolecenie z tym samym kryterium i opcjonalnym `--plot`. Wypisz podsumowanie w konsoli. Gdy podano ścieżkę PNG, zapisz również wykres. Zachowaj `--data` i podgląd startera.

W obu podpoleceniach dodaj do `argparse` opcjonalne filtry `--type` i `--from-year`. Sprawdź ich wartości i przekaż je do funkcji analizy.

### Student 3 - pliki wynikowe

1. **Eksport CSV.** Uzupełnij `write_report`. Użyj UTF-8 i nagłówka `NORAD_CAT_ID,OBJECT_NAME,OBJECT_TYPE`. Przy pustej tabeli zapisz sam nagłówek.
2. **Wykres PNG.** Uzupełnij `write_plot` w Matplotlib. Na osi X pokaż typ obiektu, a na osi Y liczbę rekordów lub startów w danej kategorii. Podpisz obie osie i zapisz wykres pod wskazaną ścieżką.

Wpiszcie nazwiska i polecenia do README. Każdy wnosi obie funkcjonalności na własnej gałęzi. Przed scaleniem ktoś z zespołu przegląda zmianę. Skonfigurujcie Poetry według instrukcji na następnej stronie.

## Oczekiwany rezultat i zaliczenie

Prowadzący klonuje oddane repozytorium i uruchamia poniższe polecenia. Sprawdźcie, czy każdy wiersz wyniku odpowiada jednemu pasującemu obiektowi oraz czy wysokości słupków odpowiadają licznikom w podsumowaniu.

```
poetry install
poetry run python main.py --help
poetry run python main.py report --name ISS --output wynik.csv
poetry run python main.py summary --name ISS --plot wykres.png
```

Oddajcie adres repozytorium z działającą wersją na gałęzi głównej, historią wkładu zespołu i plikiem danych.

# Szczegóły wykonania

## Student 1 - dane i reguły analizy

- `analyze(records, criterion, filters=None)` i `summarize(records, criterion, filters=None)` dostają listę słowników z `read_catalog`, kryterium z CLI i opcjonalne filtry. Obie zwracają osobne tabele pandas. Raport ma kolumny `NORAD_CAT_ID,OBJECT_NAME,OBJECT_TYPE`, a podsumowanie kolumny podane niżej.
- **Walidacja i wybór.** `--name` przyjmuje niepusty fragment nazwy. Dopasuj dosłowny tekst w dowolnym miejscu nazwy, używając porównania bez rozróżniania wielkości liter. W obu funkcjach stosuj wszystkie aktywne filtry.
- **Zawartość wyniku.** Zwróć pasujące wiersze w kolejności wejściowej. Podsumowanie wypisz jako `OBJECT_TYPE,COUNT`, po jednym wierszu na typ, alfabetycznie.

## Student 2 - zachowanie poleceń

- `report` wymaga kryterium i `--output SCIEZKA`. Wywołaj `analyze`, przekaż tabelę do `write_report`, a po zapisie wypisz ścieżkę do CSV.
- `summary` wymaga tego samego kryterium. Wypisz nagłówek i wiersze podsumowania. Jeśli podano `--plot SCIEZKA`, przekaż wynik do `write_plot`. Bez tej opcji polecenie wypisuje tylko tekst.
- **Dane i błędy.** Opcję globalną `--data SCIEZKA` podaje się przed podpoleceniem. Bez niej użyj pliku z katalogu `data/`. Błędny argument, brak pliku wejściowego lub błąd zapisu ma dać komunikat na stderr i niezerowy kod wyjścia.
- **Filtry dodatkowe.** Dodaj obie opcje w `report` i `summary`, sprawdź argumenty według następnej strony i przekaż słownik `filters` do funkcji studenta 1. Użyj kluczy argparse, np. `min_period` dla `--min-period`. Brak opcji przekaż jako `None`.

## Student 3 - format plików

- **CSV.** Zapisz kolumny w kolejności z pierwszej strony, z przecinkiem jako separatorem i bez indeksu pandas. Użyj ścieżki przekazanej do funkcji. Zwróć sterowanie do CLI po zapisaniu pliku.
- **PNG.** Narysuj wykres słupkowy w kolejności z podsumowania. Przy pustej tabeli zapisz wykres z podpisami osi i napisem „Brak danych”. Po zapisie zamknij figurę.

## Konfiguracja Poetry

Skonfigurujcie projekt w dostarczonym `pyproject.toml`. Uzupełnijcie opis i autorów oraz sprawdźcie zakres wersji Pythona.

W terminalu, w katalogu zawierającym `pyproject.toml`, dodajcie wymagane biblioteki:

```
poetry add pandas matplotlib
```

Aby dodać kolejny pakiet, wpiszcie `poetry add NAZWA_PAKIETU`, np. `poetry add numpy`, jeśli używacie NumPy bezpośrednio w kodzie. Polecenie instaluje pakiet i aktualizuje `pyproject.toml` oraz `poetry.lock`. Zapiszcie oba pliki w Git. Po sklonowaniu repozytorium zależności instaluje się poleceniem `poetry install`.

Zakres konfiguracji znajdziecie w dokumentacji https://python-poetry.org/docs/pyproject/ oraz w wykładzie dostępnym na Classroomie.

# Przykładowe uruchomienie

## Opcje main.py

`--data` wskazuje CSV przed podpoleceniem. Oba podpolecenia przyjmują kryterium `--name` i poniższe filtry. `report` wymaga `--output`, a `summary` zapisuje wykres po podaniu `--plot`.

`--type` przyjmuje PAY, DEB, R/B lub UNK i wymaga zgodności z OBJECT_TYPE. `--from-year` przyjmuje czterocyfrowy rok i zachowuje obiekty wystrzelone od 1 stycznia tego roku włącznie. Przy aktywnym filtrze roku pomiń brakującą lub błędną datę. Dopuszczalne lata to 1000-9999. Przy aktywnym progu kąta pomiń wiersz z błędnym kątem, chyba że zadanie wymaga błędu dla wskazanego ID.

Wklej całe polecenie do PowerShella w katalogu repozytorium. Znak na końcu pierwszego wiersza kontynuuje polecenie. Wielokropek oznacza dalsze wiersze wyniku.

## Raport CSV

```
poetry run python main.py --data data/satcat.csv report --name ISS `
    --type PAY --from-year 1990 --output wynik.csv
```

Konsola wypisuje ścieżkę `wynik.csv`. Kolejność kolumn w pliku to `NORAD_CAT_ID,OBJECT_NAME,OBJECT_TYPE`. Jeden wiersz na dopasowany obiekt, w kolejności z wejścia. W tym uruchomieniu plik ma **42 wierszy danych** oraz nagłówek.

```
NORAD_CAT_ID,OBJECT_NAME,OBJECT_TYPE
25544,ISS (ZARYA),PAY
25575,ISS (UNITY),PAY
26400,ISS (ZVEZDA),PAY
26700,ISS (DESTINY),PAY
35932,SWISSCUBE,PAY
...
```

## Podsumowanie i wykres

```
poetry run python main.py --data data/satcat.csv summary --name ISS `
    --type PAY --from-year 1990 --plot wykres.png
```

Kolejność kolumn w konsoli to `OBJECT_TYPE,COUNT`. Jeden wiersz na typ obiektu, alfabetycznie według kodu. Oczekiwany tekst:

```
OBJECT_TYPE,COUNT
PAY,42
```

## Układ wykresu PNG

Program zapisuje pionowy wykres słupkowy w `wykres.png`.

- **Tytuł.** Obiekty według typu
- **Oś X.** „Typ obiektu”. Etykiety pochodzą z pierwszej kolumny podsumowania i zachowują kolejność jego wierszy.
- **Oś Y.** „Liczba obiektów”. Wysokości słupków pochodzą z drugiej kolumny podsumowania. Skala zaczyna się od zera.